CampusConnect

A Web-Based Student Portal for College Academics, Notices, Events, Resources and Support

Project Details Document (source material for the project report)

Field	Details
Programme	B.Tech — Computer Science & Engineering
University	Maharaja Ranjit Singh Punjab Technical University (MRSPTU), Bathinda
Project type	Full-stack web application (practical / viva submission)
Student name	[Your Name]
Roll number	[Your Roll No.]
Guide / Supervisor	[Guide Name, Designation]
Session	[2025-26]

Note for the report writer: This document contains every technical fact about the project — objectives, architecture, modules, database design, API, security, testing and screenshots. Use it to write a formal B.Tech project report (Certificate, Declaration, Acknowledgement, Abstract, chapters, references). Replace the bracketed placeholders with the student's details.

1. Abstract

CampusConnect is a role-based web portal that brings a college student's day-to-day information into one organised website. Students sign in to view their subjects, weekly timetable, assignments, department notices, club events, campus resources and to raise support requests whose status they can track. An administrator (Head of Department) signs in to a separate console to publish and manage notices, events and resources, and to review and resolve student requests. Both portals read and write the same live database, so every change made by the administrator is visible to students immediately and every student action (event registration, support request, assignment submission) is visible to the administrator.

The system is built with a React + TypeScript single-page frontend, a Python FastAPI REST backend and a MongoDB database, with cookie-based session authentication, server-side validation and an automated test suite.

2. Introduction & Problem Statement

In most colleges, student information is scattered: notices are pinned on physical boards or forwarded as PDFs on messaging groups, timetables are shared as images, event registrations happen on separate forms, and complaints are raised verbally or on paper with no way to track them. Students miss deadlines and announcements, and the department has no single place to publish information or see what students need.

Problem: There is no single, organised, always-available platform where students can access academic and campus information and where the department can publish and manage that information.

Solution: CampusConnect — a centralised, responsive web portal with separate student and administrator roles, backed by a shared database so information stays consistent for everyone.

3. Objectives

- Provide a single portal for academics, notices, events, resources and student support.
- Implement secure login with two roles — Student and Administrator — with role-based access to pages and APIs.
- Give students a dashboard summarising today's classes, pending assignments, latest notices and upcoming events.
- Allow the administrator to create, edit, publish/unpublish and delete notices, events and resources (CRUD).
- Provide a support/grievance system where students raise requests and track status (Pending → In review → Resolved).
- Keep data synchronised in real time between the student and administrator portals through one database.
- Build a responsive, accessible interface that works on desktop and mobile, including a dark mode.
- Validate all inputs on both client and server and verify the system with automated tests.

4. Scope

4.1 In scope

- Public landing page with portal overview, live notice and event preview, and login entry points.

- Student module: dashboard, academics (subjects, timetable, assignments), notices, events & clubs, resources, support, profile & settings.
- Administrator module: dashboard statistics and recent activity, notice/event/resource management, support-ticket management.

4.2 Out of scope (future work)

- Integration with the university ERP / real attendance systems.
- E-mail or SMS notifications, online fee payment, file uploads, a native mobile app, separate faculty role.

5. Technology Stack

Layer	Technology	Purpose
Frontend	React 19, TypeScript (strict), Vite	Single-page application, type-safe UI code, fast builds
Styling / UI	Tailwind CSS v4, shadcn/ui components, Lucide icons	Responsive layout, consistent design system, dark mode
Data fetching	TanStack Query (React Query)	Server-state caching, automatic refetch and cache invalidation after changes
Routing	React Router	Client-side routes for 14 pages, protected (role-guarded) routes
Animation	Motion (Framer Motion)	Subtle page and list transitions
Notifications (UI)	Sonner	Success / error toast messages
Backend	Python 3.11, FastAPI, Uvicorn	REST API, request validation, async I/O
Validation	Pydantic v2	Typed request/response models with length and value rules
Database	MongoDB (Motor async driver)	Document storage for users, notices, events, tickets, etc.
Auth	PBKDF2-SHA256 password hashing, HTTP-only session cookies	Secure login without storing plain passwords
Testing	Pytest + HTTPX (API), Playwright (browser)	Automated backend and end-to-end UI tests
Version control / Deploy	Git, GitHub, Render (+ MongoDB Atlas)	Source control and cloud hosting

6. System Architecture

CampusConnect follows a three-tier client–server architecture:



- The browser calls only relative `/api/...` paths through one typed fetch layer; the backend is the single source of truth.
- After any create/update/delete, the frontend invalidates the related cached queries so screens refetch fresh data — this is how admin changes appear in the student portal (and vice versa).
- All dates are anchored on the server in Indian Standard Time (Asia/Kolkata) so “today's timetable” and “upcoming events” are correct regardless of the viewer's device.
- On first start with an empty database, the server seeds demo accounts and sample departmental data; an existing database is never overwritten.

7. Modules & Features

7.1 Public module

- Landing page: hero section, feature overview, live preview of latest notices and upcoming events, demo login details.
- Login page: Student / Administrator tabs, e-mail + password form, error messages for wrong credentials.

7.2 Student module

Page	Features
Dashboard	Greeting with roll number & semester; cards for attendance, CGPA, pending assignments, upcoming events; today's timetable (IST); pending assignments; latest notices; upcoming events; quick links to university portals.
Academics	Tabs: Subjects (credits, faculty, attendance % and syllabus progress bars, viva readiness), Weekly Timetable (today highlighted), Assignments (pending / submitted / graded, “Mark submitted” action, average marks).
Notices	Search box, category filters (Academic, Examination, Viva Voce, Placement, Hostel, Sports), list with dates, detail dialog with full text. Only published notices are visible.
Events & Clubs	Club cards (Coding, Robotics, CSI, ISTE); upcoming and past events with date, venue, capacity; one-click Register / Cancel with live registration count.
Resources	Syllabus, lab manuals, previous papers, academic calendar, policies, links — filter by type; important contacts; FAQ accordion.
Support	Form (category, subject, priority, description) with validation; list of own requests with status badge and update timeline (who changed it, note, time).
Profile & Settings	Academic details, viva status, notification preferences, light/dark theme, sign out.

7.3 Administrator module

Page	Features
Admin Dashboard	Statistics: notices published / total, active grievances, upcoming events, students, resources; quick actions; recent notices, upcoming events and latest support requests.
Manage Notices	Table of all notices incl. drafts; search; create / edit dialog (title, category, body, publish now); publish/unpublish toggle; delete with confirmation.
Manage Events	Table with club, date, venue, RSVPs (registered/capacity); create / edit (date-time, capacity); delete (also removes registrations).
Manage Resources	Table; create / edit (title, type, subject, URL, description); delete with confirmation.
Student Helpdesk	All tickets with filters (All / Pending / In review / Resolved); open a ticket to see description and timeline; change status and add a note visible to the student.

7.4 Data synchronisation between portals

Action	Done by	Reflected in
Create / edit / publish / delete notice	Admin	Student notices page, student dashboard, landing page
Create / edit / delete event	Admin	Student events page, dashboard, landing page
Register / cancel for event	Student	Admin events table (RSVP count), event cards
Add / edit / delete resource	Admin	Student resources page
Raise support request	Student	Admin helpdesk, admin dashboard statistics
Change ticket status + note	Admin	Student support page (status badge and timeline)
Mark assignment submitted	Student	Assignment status on academics & dashboard

8. Database Design (MongoDB collections)

Collection	Key fields	Notes
users	id, name, email (unique), password_hash, role (student | admin), roll_no, branch, semester, batch, cgpa, attendance, designation	Passwords stored only as salted PBKDF2 hashes
sessions	token (unique), user_id, created_at	One row per login; deleted on logout
subjects	code (unique), name, credits, faculty, attendance_pct, progress_pct, viva_status	CS-801 … CS-805
timetable	day, day_index, start, end, subject, code, room, faculty, kind	Lecture / Lab / Tutorial / Seminar
assignments	id, title, code, subject, due, status (pending | submitted | graded), marks, submitted_at	
notices	id, title, category, body, author, published, created_at	Drafts hidden from students
events	id, title, club, category, description, venue, date, capacity	Registration count computed per request
event_registrations	id, event_id, student_id, student_name, registered_at	Unique (event_id, student_id) — no double booking
resources	id, title, kind, subject, url, description, created_at	
support_tickets	id, student_id, student_name, roll_no, category, subject, priority, description, status, created_at, updates[]	updates[] = {status, note, at, by} timeline

Indexes are created at start-up on every field used for lookups, sorting or uniqueness (e.g. users.email unique, notices by published + created_at, events by date, registrations unique per student per event).

8.1 Entity relationships

```
USER (student) 1 ---- * SUPPORT_TICKET 1 ---- * TICKET_UPDATE (embedded)
USER (student) 1 ---- * EVENT_REGISTRATION * ---- 1 EVENT
USER (admin)   1 ---- * NOTICE   (author)
USER           1 ---- * SESSION
SUBJECT        1 ---- * TIMETABLE_ENTRY / ASSIGNMENT (by subject code)
```

9. REST API Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/login	Public	Verify e-mail/password, set session cookie

Method	Endpoint	Access	Description
POST	/api/auth/logout	Signed in	Delete session, clear cookie
GET	/api/auth/me	Public	Current user or null
GET	/api/academics/subjects	Signed in	Subject list
GET	/api/academics/timetable	Signed in	Full weekly timetable
GET	/api/academics/today	Signed in	Today's classes (IST)
GET	/api/academics/assignments	Signed in	Assignments by due date
POST	/api/academics/assignments/{id}/submit	Signed in	Mark pending assignment submitted
GET	/api/notices?search=&category=&all=	Public	Published notices; all=1 adds drafts for admin
POST / PUT / DELETE	/api/notices[/{id}]	Admin	Create / update / delete notice
GET	/api/events?scope=upcoming|all	Public	Events with registration count and status
POST / DELETE	/api/events/{id}/register	Student	Register / cancel (blocked if past or full)
POST / PUT / DELETE	/api/events[/{id}]	Admin	Create / update / delete event
GET	/api/resources?kind=	Public	Resource list
POST / PUT / DELETE	/api/resources[/{id}]	Admin	Create / update / delete resource
POST	/api/support	Student	Raise support request
GET	/api/support/mine	Signed in	Own requests
GET	/api/support?status=	Admin	All requests, optional status filter
PATCH	/api/support/{id}	Admin	Change status, add note to timeline
GET	/api/admin/stats	Admin	Dashboard statistics

10. Security & Validation

- **Password hashing:** PBKDF2-HMAC-SHA256 with a random 16-byte salt and 100,000 iterations; constant-time comparison.
- **Sessions:** random 256-bit token in an HTTP-only, SameSite=Lax cookie (not readable by JavaScript), valid 7 days, removed on logout.
- **Role-based access control:** server-side dependencies *require_user* and *require_admin*; unauthenticated requests get 401, students calling admin APIs get 403. The frontend also guards routes (students are redirected away from /admin).
- **Input validation:** Pydantic models enforce required fields, maximum lengths, allowed values (ticket status, priority) and numeric ranges (event capacity); invalid data returns 422. Forms disable submit until required fields are filled.
- **Business rules:** no registration for past or full events; unique registration per student per event; only pending assignments can be submitted; students see only their own support tickets; drafts are never exposed to students.
- **Safe search:** search text is regex-escaped before querying the database.

11. Key Workflows

11.1 Login

User opens /login → selects Student or Administrator → submits e-mail and password → server verifies the hash → creates a session and sets the cookie → frontend clears any old cached data and redirects to /dashboard (student) or /admin (admin).

11.2 Support request lifecycle

```
Student submits form --> ticket created (status: pending, timeline: "Request received")
      |
Admin opens ticket in Helpdesk --> sets "In review" + note --> timeline entry added
      |
Admin sets "Resolved" + note --> student sees resolved badge and the note
```

11.3 Notice publishing

Admin creates a notice (published or draft) → it is stored with author and IST timestamp → published notices appear instantly for students with search and category filtering → admin may later edit, unpublish or delete it.

11.4 Event registration

Student opens Events → clicks Register on an upcoming event → server checks the event exists, is not in the past and is not full → registration saved → count updates for the student and in the admin events table. Cancel removes it.

12. Testing

The application was tested with automated backend API tests (Pytest + HTTPX), browser end-to-end tests (Playwright) and manual API checks (curl). Final result: **12/12 backend tests passed and all 7 browser test scenarios passed**, with no console errors or failed requests.

#	Test case	Expected result	Status
1	Login with valid student / admin credentials	Redirect to correct dashboard	Pass
2	Login with wrong password	401 + error message	Pass
3	Logged-out user opens /dashboard or /admin	Redirected to /login	Pass
4	Student opens /admin or calls /api/admin/stats	Redirected / 403 Forbidden	Pass
5	Admin creates, edits, unpublishes, deletes a notice	Student view updates at each step; drafts hidden	Pass
6	Admin creates, edits, deletes an event	Reflected on student events page	Pass
7	Student registers / cancels for an event	RSVP count updates in admin table	Pass
8	Register for a past event	400 — event already took place	Pass
9	Admin adds / deletes a resource	Student resources page updates	Pass
10	Student raises support request	Appears in admin helpdesk	Pass
11	Admin resolves ticket with note	Student sees Resolved + note in timeline	Pass
12	Submit request with empty subject / invalid status	422 validation error	Pass
13	Mark assignment submitted	Status changes to submitted	Pass
14	All student pages load	No console errors / failed requests	Pass
15	Dark-mode toggle	Persists after reload	Pass

12.1 Bug found and fixed during testing

Testing revealed that creating a new event stored computed display fields in the database, which caused the events list to fail with a server error. The fix stores only real event fields and recomputes per-viewer values (registered, count, past) on every request; the regression tests now pass.

13. Screenshots

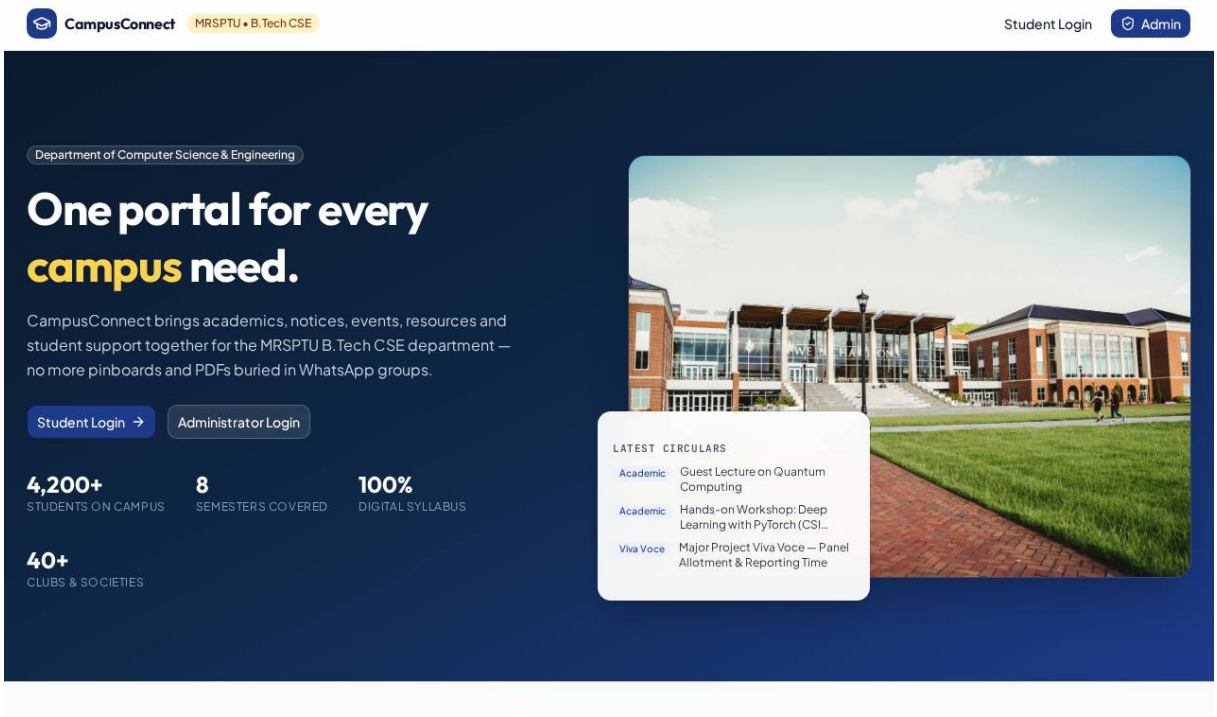


Fig 13.1 — Public landing page

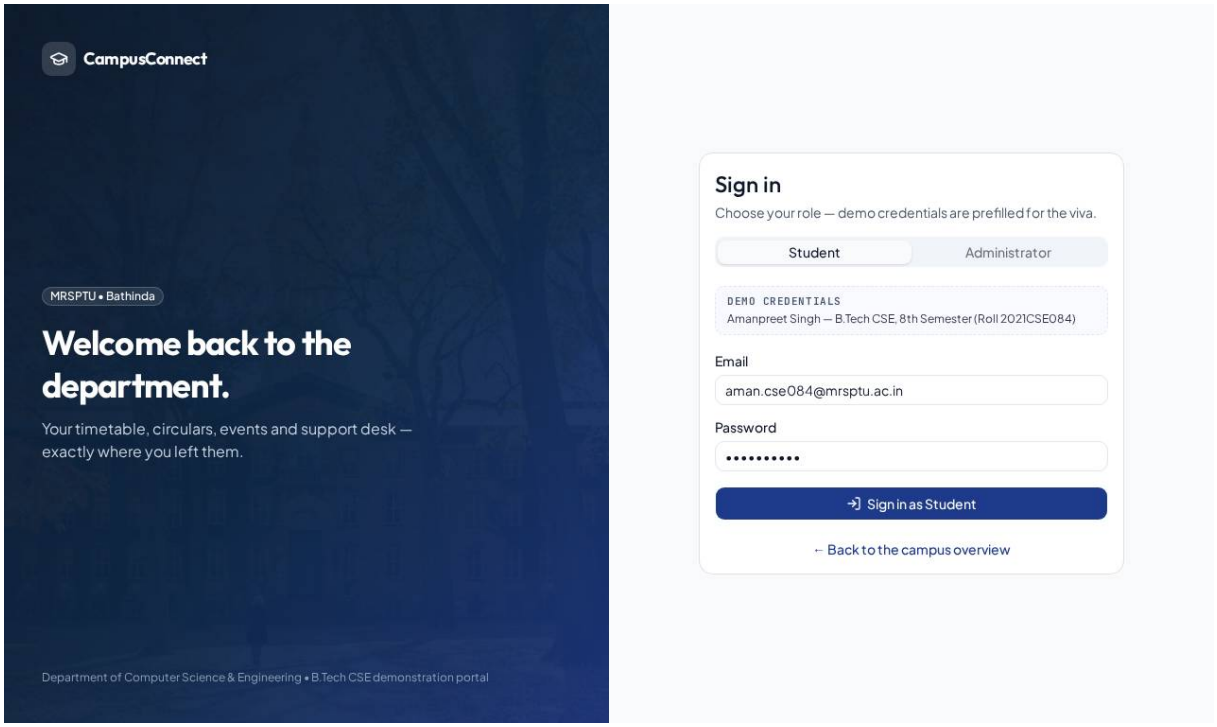


Fig 13.2 — Login with Student / Administrator roles

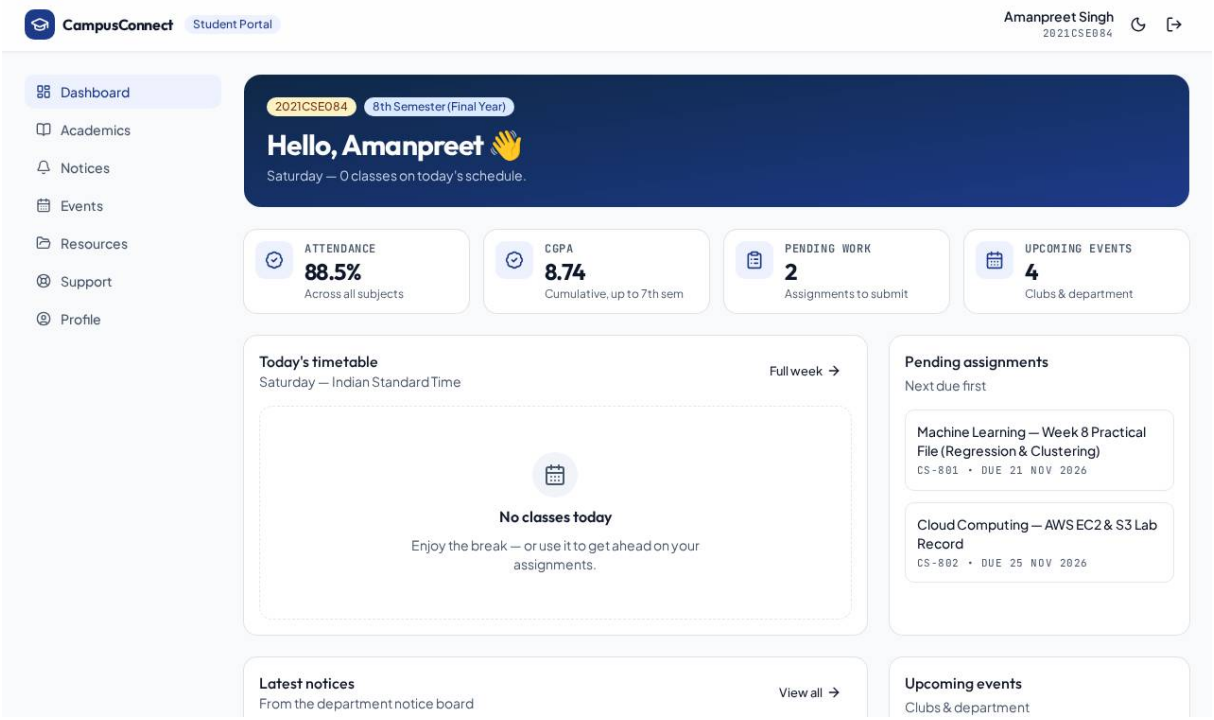


Fig 13.3 — Student dashboard

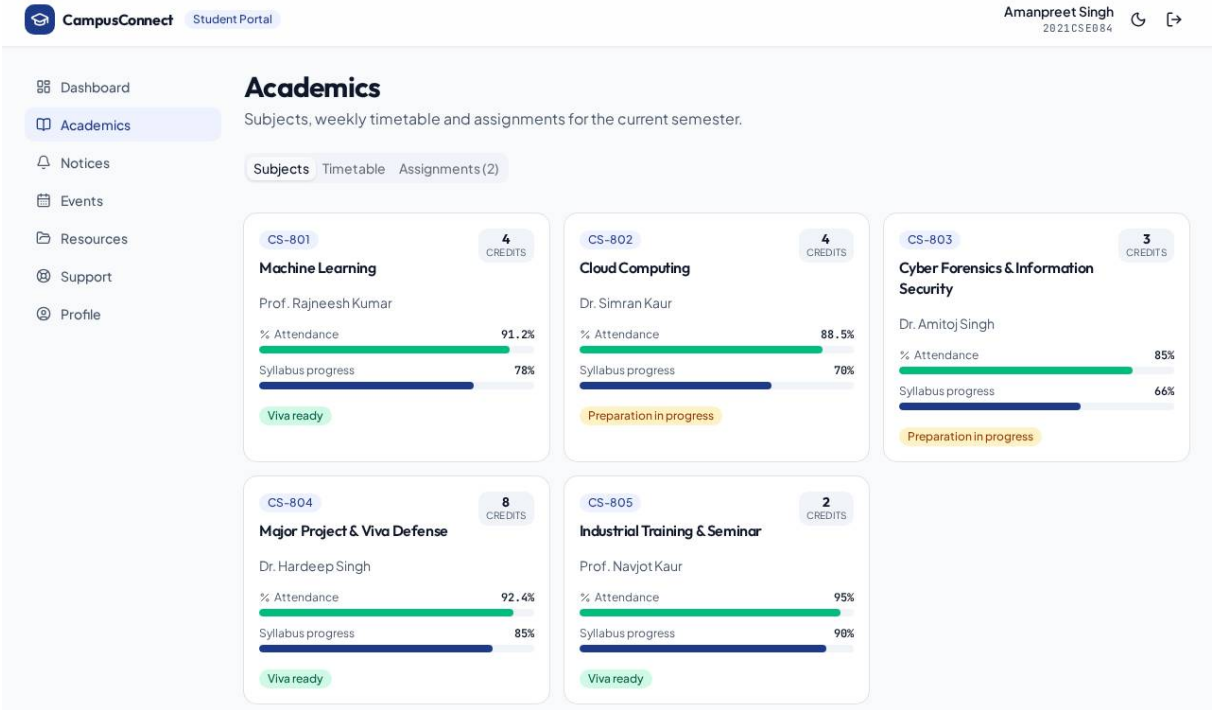


Fig 13.4 — Academics: subjects with attendance and progress

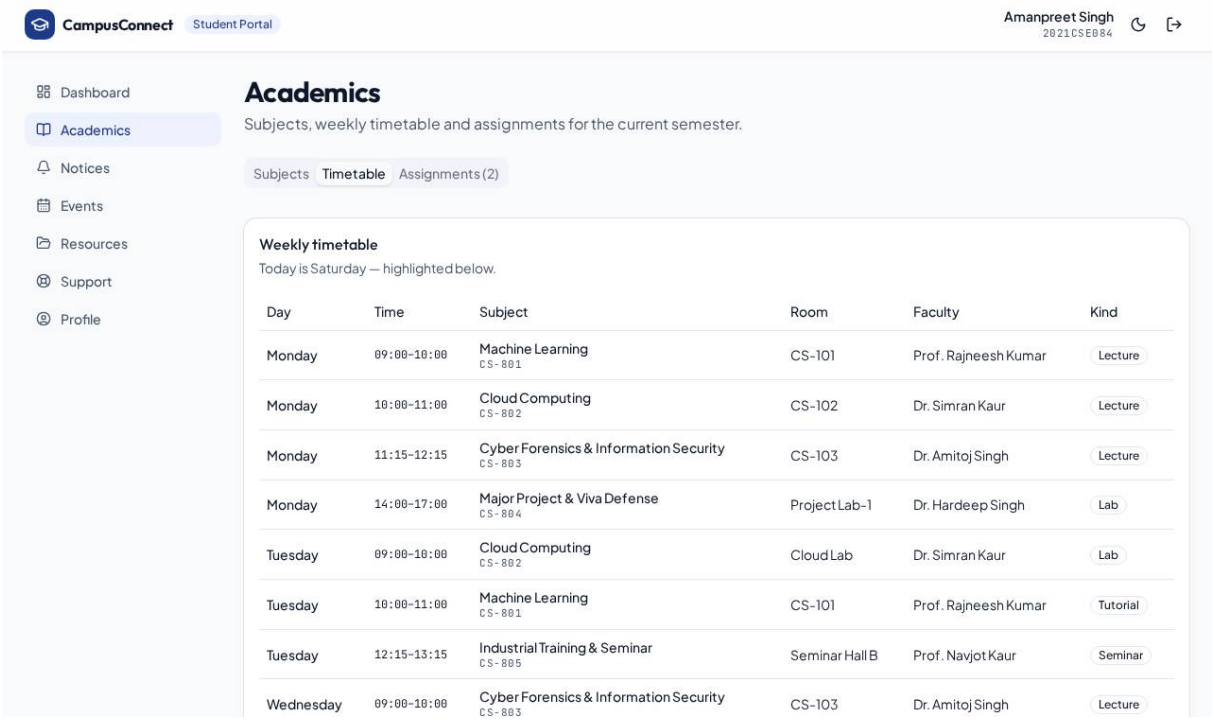


Fig 13.5 — Academics: weekly timetable

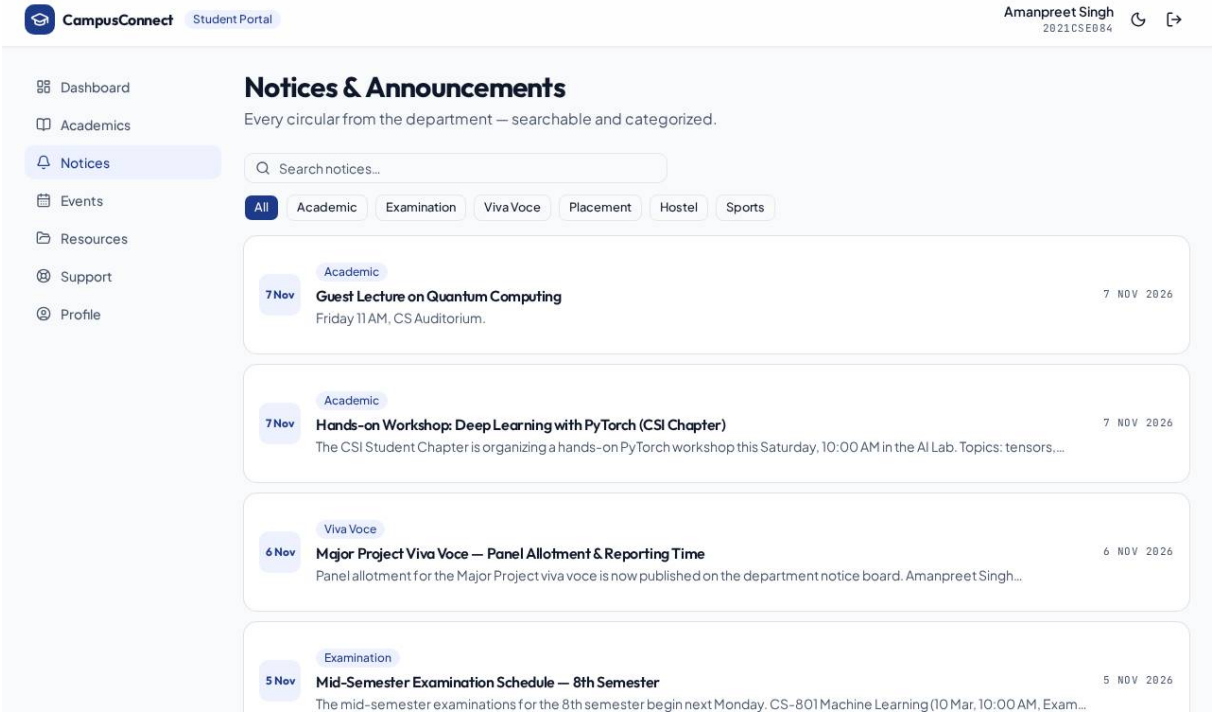


Fig 13.6 — Notices with search and category filters

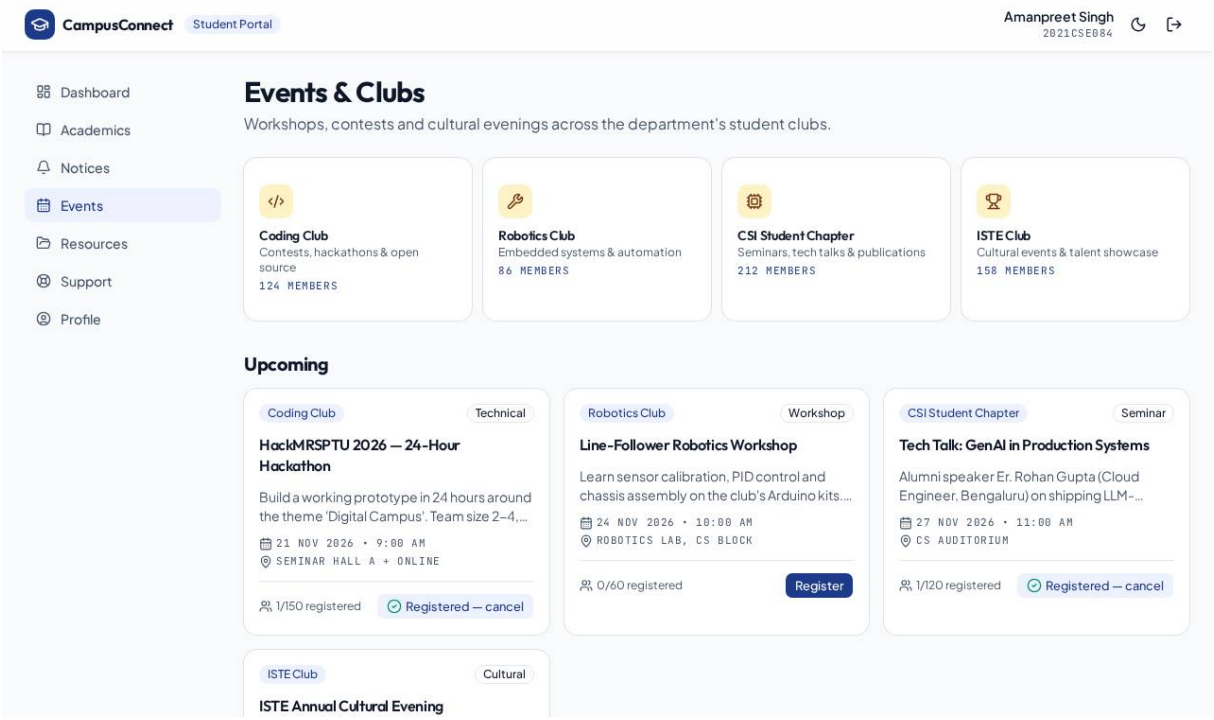


Fig 13.7 — Events & clubs with registration

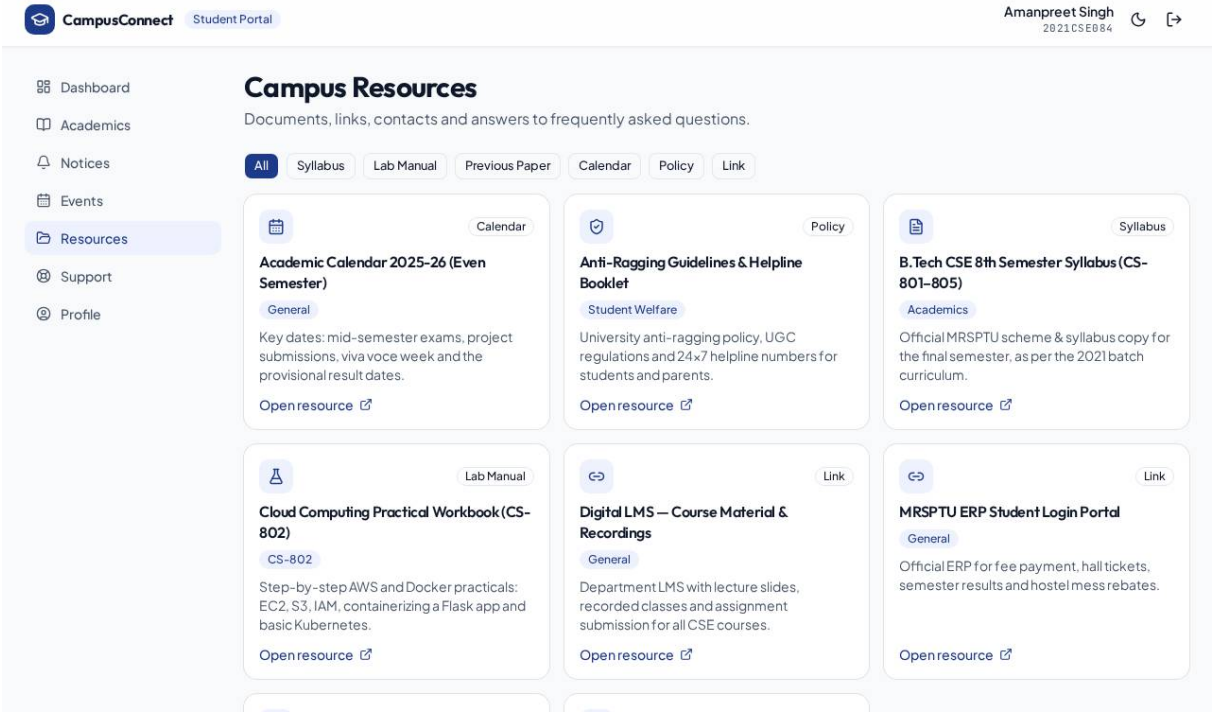


Fig 13.8 — Campus resources

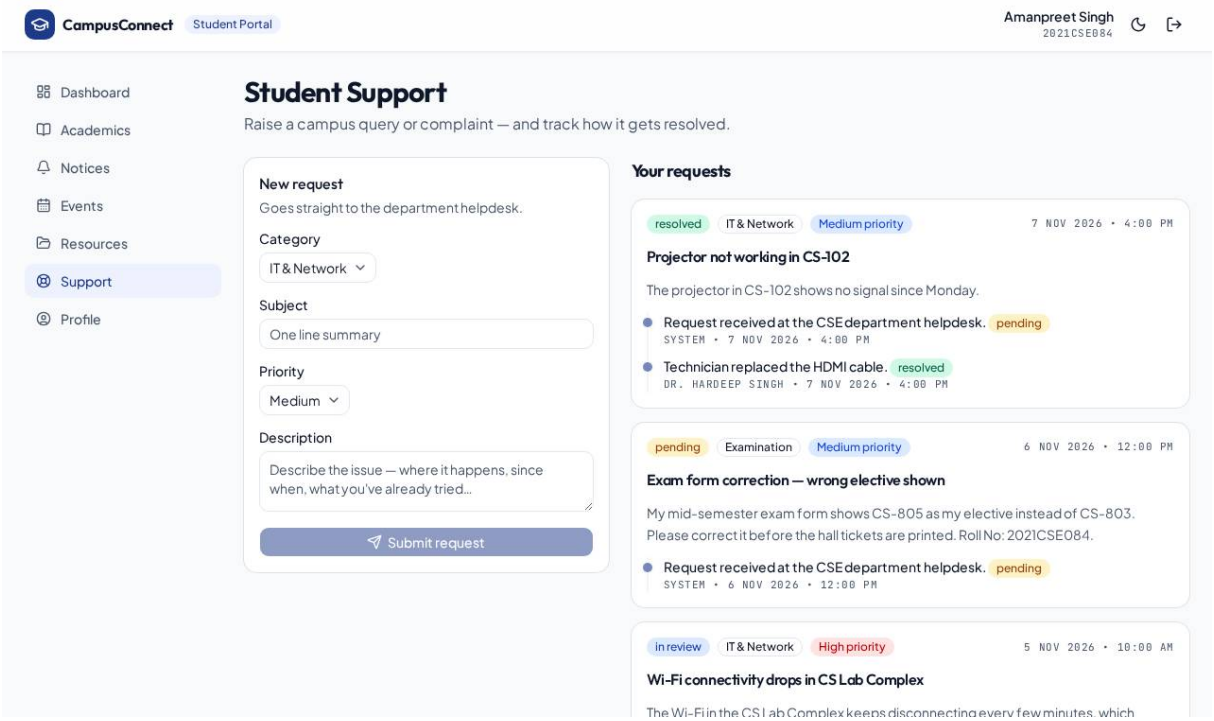


Fig 13.9 — Student support request and status timeline

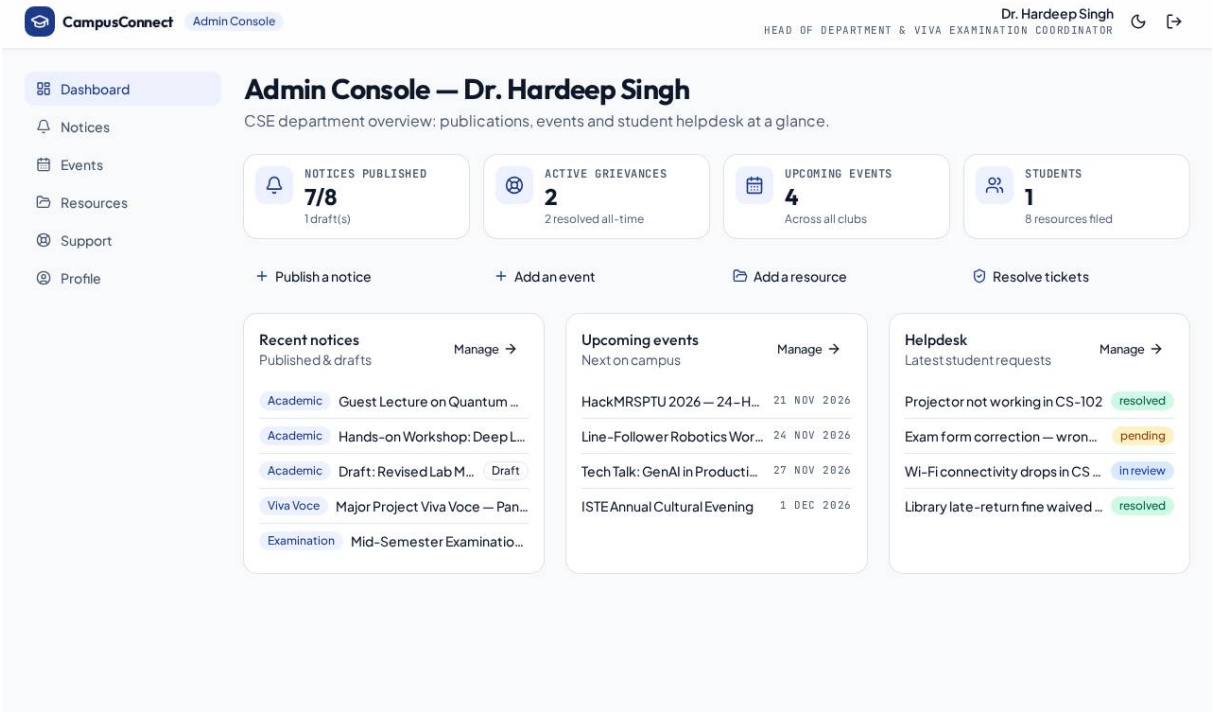


Fig 13.10 — Administrator dashboard

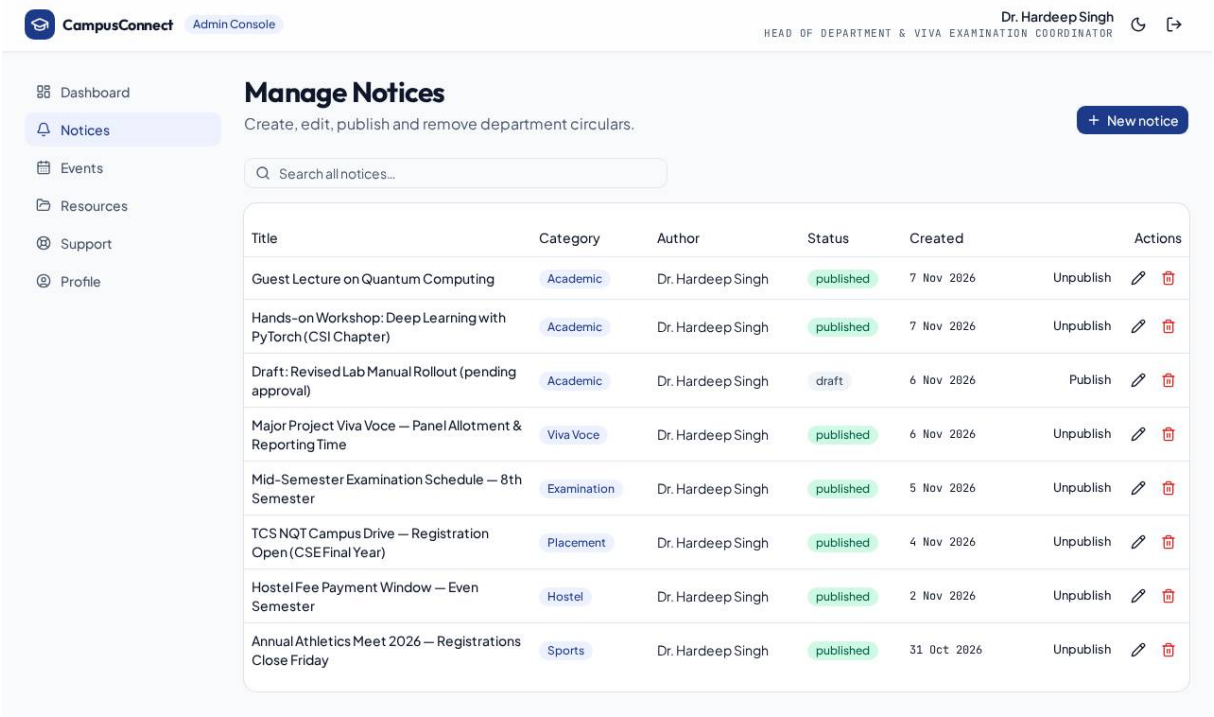


Fig 13.11 — Administrator: manage notices

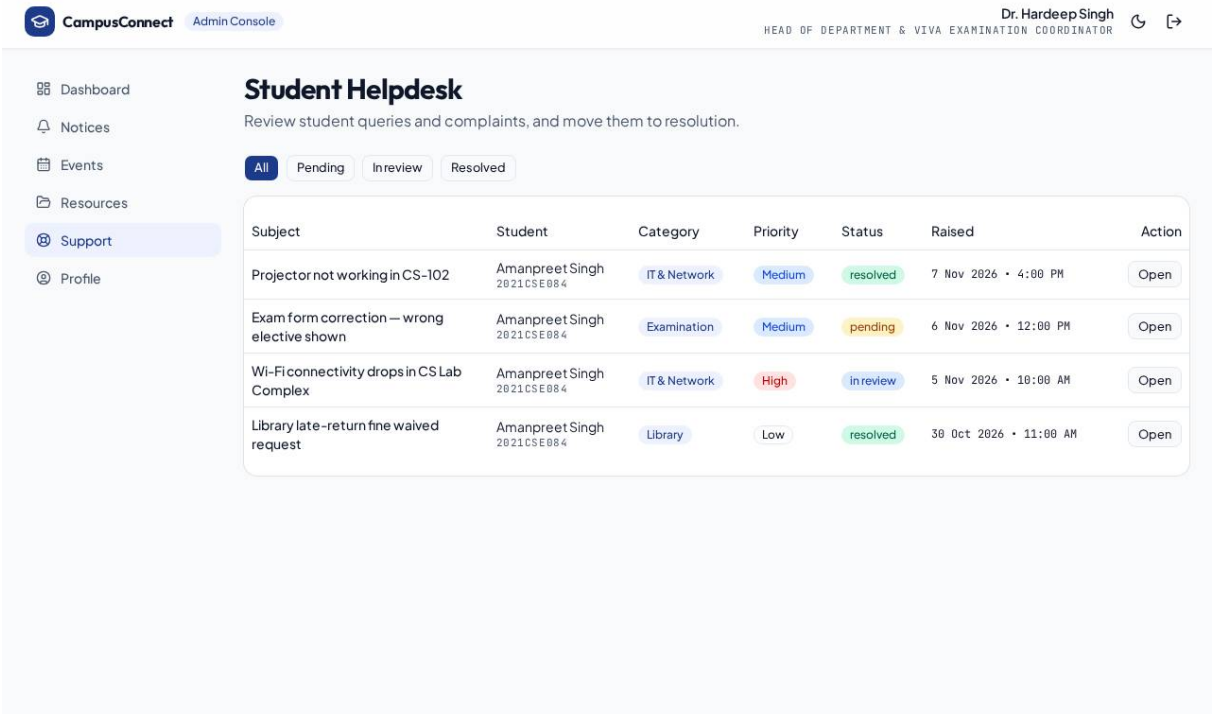


Fig 13.12 — Administrator: student helpdesk

14. Project Structure

```
campusconnect/
■■■ backend/                    FastAPI application
■   ■■■ server.py               app, CORS, router registration, startup (indexes + auto-seed)
■   ■■■ seed.py                 idempotent sample data (users, subjects, timetable, notices...)
■   ■■■ lib/  auth.py           password hashing, sessions, role guards
■   ■         db.py             MongoDB connection + indexes
■   ■         dates.py          IST date helpers
■   ■■■ models/                 Pydantic models (auth, academics, portal, support)
■   ■■■ routers/                auth, academics, notices, events, resources, support, admin
■   ■■■ tests/                  pytest API tests
■   ■■■ requirements.txt
■■■ frontend/                   React + TypeScript (Vite)
■   ■■■ index.html
■   ■■■ src/
■       ■■■ App.tsx             routes (public, student, admin)
■       ■■■ components/         AppShell (nav), RequireAuth (route guard), shared UI bits
■       ■■■ lib/                api.ts (typed fetch), session.ts, types.ts, format.ts, theme.ts
■       ■■■ pages/              Home, Login, Dashboard, Academics, Notices, Events, Resources,
■                               Support, Profile, AdminDashboard, AdminNotices, AdminEvents,
■                               AdminResources, AdminSupport
■■■ render.yaml                 one-click Render deployment blueprint
■■■ DEPLOY_RENDER.md            deployment guide
```

15. Deployment

- Source code hosted on GitHub.
- Database: MongoDB Atlas (free M0 cluster) — connection string supplied as the MONGO_URL environment variable.
- Backend: Render Web Service — `pip install -r requirements.txt`, start `uvicorn server:app --host 0.0.0.0 --port $PORT`.
- Frontend: Render Static Site — `yarn build`, publish `dist`; `/api/*` is rewritten to the backend so login cookies remain same-origin.
- On first launch the backend creates the demo accounts and sample data automatically.

16. Demo Credentials

Role	E-mail	Password	Profile
Student	aman.cse084@mrsptu.ac.in	student123	Amanpreet Singh, Roll 2021CSE084, B.Tech CSE 8th Sem
Administrator	hod.cse@mrsptu.ac.in	admin123	Dr. Hardeep Singh, Head of Department, CSE

17. Advantages & Limitations

Advantages

- Single source of truth — no outdated copies of notices or timetables.
- Role-based security on both server and client.
- Responsive design usable on phones; dark mode for comfort.
- Transparent grievance handling with a full status timeline.
- Modular code (separate routers, models and pages) that is easy to extend.

Limitations

- Uses sample academic data rather than the live university ERP.
- Single administrator role; no separate faculty accounts yet.
- No e-mail/SMS alerts or file uploads in the current version.

18. Future Scope

- Faculty role to post notices and grade assignments for their own subjects.
- Integration with the university ERP for real attendance, results and fee status.
- E-mail / push notifications for new notices, event reminders and ticket updates.
- File uploads for assignment submissions and notice attachments.
- Analytics for the department (attendance trends, ticket resolution time).
- Progressive Web App / Android app.

19. Conclusion

CampusConnect meets its objective of providing one organised portal for students and the department. It implements secure role-based authentication, complete CRUD management for the administrator, real-time consistency between the student and administrator views through a shared database, input validation and business rules on the server, and a responsive interface. Automated backend and browser testing confirmed that all major workflows function correctly.

20. Suggested References

- FastAPI documentation — https://fastapi.tiangolo.com
- React documentation — https://react.dev
- MongoDB manual — https://www.mongodb.com/docs/manual
- TanStack Query documentation — https://tanstack.com/query
- Pydantic documentation — https://docs.pydantic.dev
- Tailwind CSS documentation — https://tailwindcss.com/docs
- OWASP Password Storage Cheat Sheet — https://cheatsheetseries.owasp.org
- Render documentation — https://render.com/docs